

TDS1 Project - Online Bookstore Database

Richard Chovanec

May 2026

Contents

1	Project Overview	3
2	Database Tables	3
3	Model Images	3
3.1	Logical Model	4
3.2	Relational Model	5
4	DD Requirements	6
4.1	DD S01 L02 - Data and Information	6
4.2	DD S02 L02 - Entities, Instances, Attributes, Identifiers	6
4.3	DD S03 L01 - Relationships, Cardinality, Optionality	6
4.4	DD S03 L02 - ER Diagram	7
4.5	DD S30 L04 - Relationship Matrix Diagram	7
4.6	DD S04 L01 - Supertypes and Subtypes	7
4.7	DD S04 L02 - Business Rules	7
4.8	DD S05 L01 - Transferable and Non-transferable Relationship	7
4.9	DD S05 L03 - M:N Relationships	7
4.10	DD S06 L01 - Identifying 1:N Relationship	7
4.11	DD S06 L02-04 - Normal Forms	7
4.12	DD S07 L01 - ARC	7
4.13	DD S07 L02 - Hierarchical and Recursive Relations	8
4.14	DD S07 L03 - Historical Data	8
4.15	DD S09 L01 - Storing Changes in Time	8
4.16	DD S09 L02 - Journaling	8
4.17	DD S10 L01 - Readability Conventions	8
4.18	DD S10 L02 - Generic Modeling	8
4.19	DD S11 L01 - Integrity Constraints	8
4.20	DD S11 L02-04 - Relational Schema Generation	8
5	SQL Requirements	8
5.1	DD S15 L01 - string concatenation with , CONCAT(), DISTINCT	8
5.2	DD S16 L02 - WHERE, LOWER, UPPER, INITCAP	9
5.3	DD S16 L03 - BETWEEN, LIKE, IN, IS NULL, IS NOT NULL	9
5.4	DD S17 L01 - AND, OR, NOT, parentheses	9
5.5	DD S17 L02 - ORDER BY ASC/DESC and multiple columns	9
5.6	DD S17 L03 - single-row and multi-row functions	9
5.7	SQL S01 L01 - character functions and DUAL	9
5.8	SQL S01 L02 - ROUND, TRUNC, MOD	9
5.9	SQL S01 L03 - date functions and SYSDATE	9
5.10	SQL S02 L01 - TO_CHAR, TO_NUMBER, TO_DATE	10
5.11	SQL S02 L02 - NVL, NVL2, NULLIF, COALESCE	10
5.12	SQL S02 L03 - DECODE, CASE, IF-THEN-ELSE	10

5.13	SQL S03 L01 - NATURAL JOIN, CROSS JOIN	10
5.14	SQL S03 L02 - JOIN USING and JOIN ON	10
5.15	SQL S03 L03 - OUTER JOIN examples	10
5.16	SQL S03 L04 - self join and hierarchical query on recursive category table	10
5.17	SQL S04 L02 - aggregate functions	11
5.18	SQL S04 L03 - COUNT variants and NVL	11
5.19	SQL S05 L01 - GROUP BY and HAVING	11
5.20	SQL S05 L02 - ROLLUP, CUBE, GROUPING SETS	11
5.21	SQL S05 L03 - UNION, UNION ALL, INTERSECT, MINUS	11
5.22	SQL S06 L01 - subqueries: scalar, multi-column, EXISTS, NOT EXISTS	11
5.23	SQL S06 L02 - single-row subquery	11
5.24	SQL S06 L03 - multi-row subqueries IN, ANY, ALL and NULL comment	12
5.25	SQL S06 L04 - WITH AS	12
5.26	SQL S07 L01 - INSERT forms	12
5.27	SQL S07 L02 - UPDATE and DELETE	12
5.28	SQL S07 L03 - DEFAULT, MERGE, multi-table insert	12
5.29	SQL S08 L01 - objects: table, index, constraint, view, sequence, synonym; CREATE/ALTER/- DROP/RENAME/TRUNCATE	12
5.30	SQL S08 L02 - datatype demo: timestamps, intervals, text, number, blob	13
5.31	SQL S08 L03 - ALTER TABLE, DROP, RENAME, FLASHBACK, DELETE, TRUNCATE, COMMENT, SET UNUSED	13
5.32	SQL S10 L01 - named constraints and CTAS	13
5.33	SQL S10 L02 - constraints including FK and CHECK	13
5.34	SQL S10 L03 - USER_CONSTRAINTS	14
5.35	SQL S11 L01 - views: FORCE, NOFORCE, WITH CHECK OPTION, READ ONLY, simple and complex	14
5.36	SQL S11 L03 - inline view	14
5.37	SQL S12 L01 - CREATE and ALTER SEQUENCE	14
5.38	SQL S12 L02 - index and key objects	14
5.39	SQL S13 L01 - GRANT and REVOKE on a view to PUBLIC	14
5.40	SQL S13 L03 - regular expressions	15
5.41	SQL S14 L01 - transaction commands	15
5.42	SQL S15 L01 - old join syntax and old outer join syntax	15
5.43	SQL S16 L03 - recap query combining several concepts	15

1 Project Overview

The project models an online bookstore. The database uses primary keys, foreign keys, check constraints, sequences, views, sample data, and SQL examples for Oracle SQL Developer.

2 Database Tables

Table	Purpose
tds_person	Supertype table for all people.
tds_customer	Customer subtype of person.
tds_employee	Employee subtype of person.
tds_employee_salary_history	Historical salary records.
tds_publisher	Book publishers.
tds_author	Book authors.
tds_category	Book category hierarchy.
tds_book	Books sold by the store.
tds_book_author	M:N relation between books and authors without extra information.
tds_customer_order	Customer orders.
tds_order_item	Order lines, also M:N relation between orders and books with extra information.
tds_payment	Payments for orders.

3 Model Images

The diagrams were exported from Oracle SQL Developer Data Modeler.

3.1 Logical Model

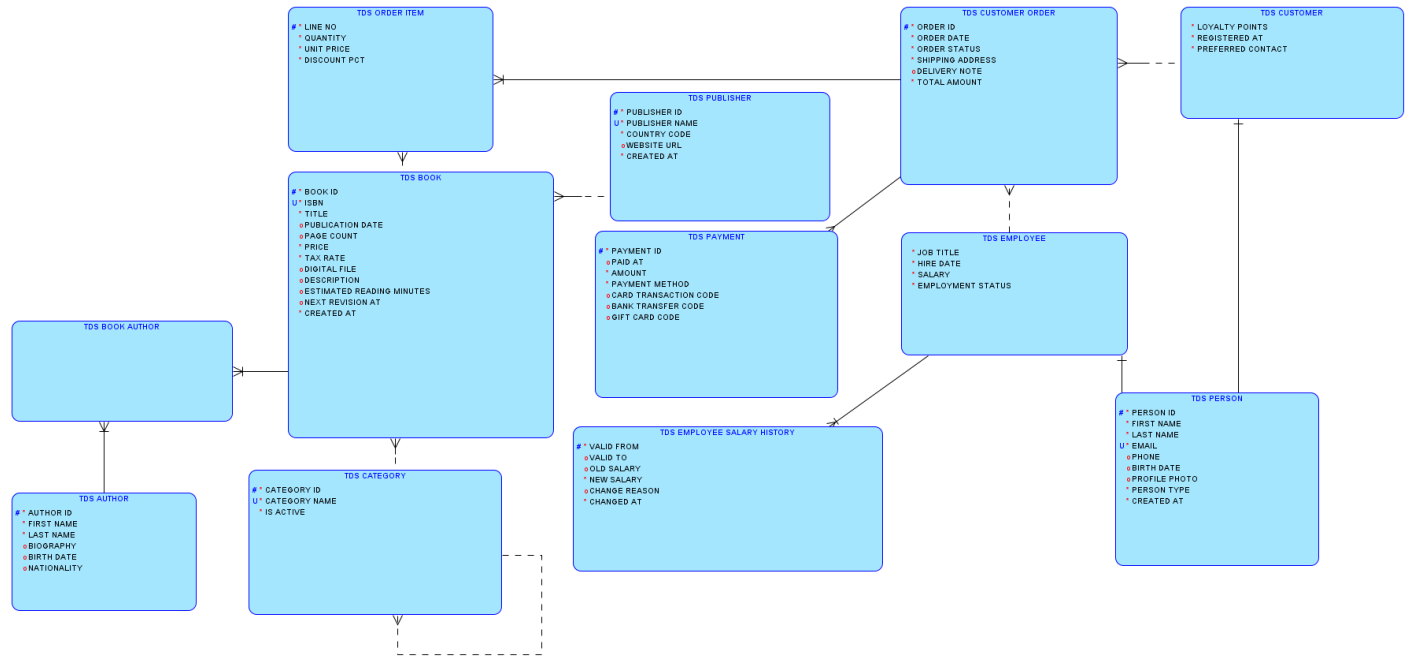


Figure 1: Logical model of the online bookstore database.

3.2 Relational Model

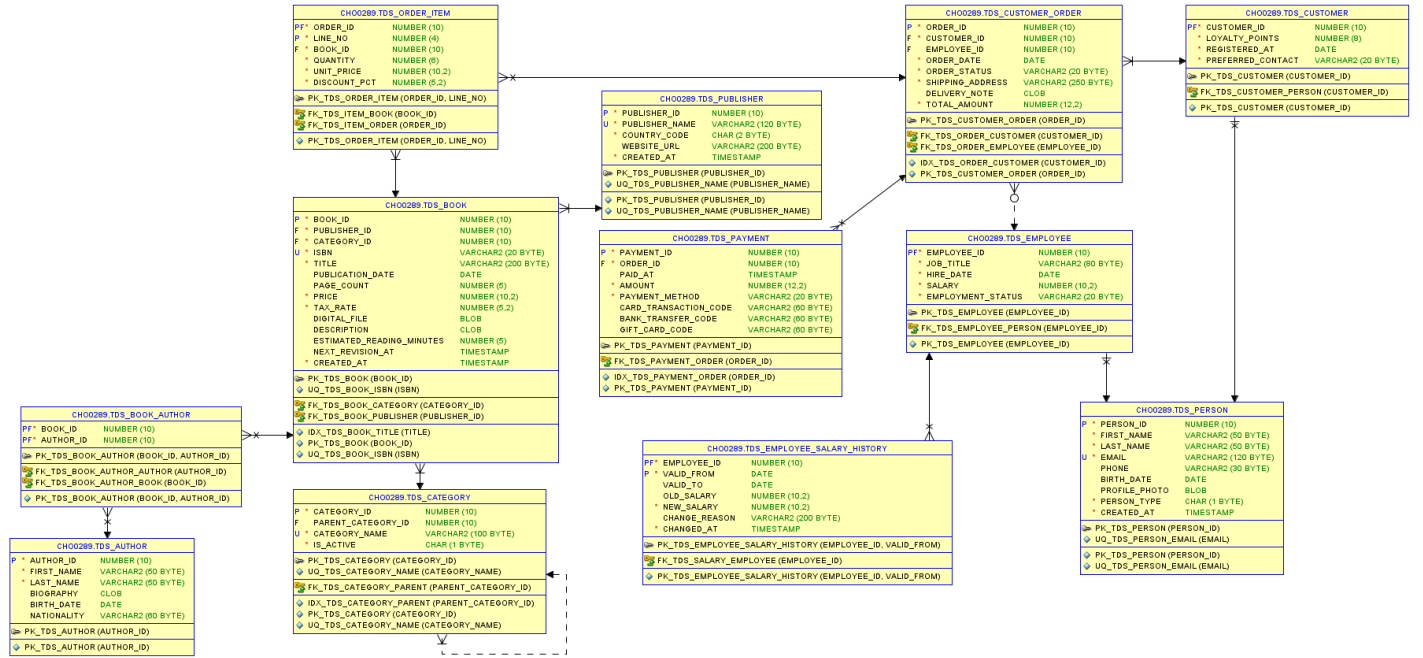


Figure 2: Relational model of the online bookstore database.

4 DD Requirements

4.1 DD S01 L02 - Data and Information

Data are raw stored values. In this project, `price = 399.90`, `quantity = 2`, and `order_date = DATE '2026-04-10'` are data. Information is the useful meaning calculated from data. For example, the information that a customer spent 799.80 CZK on two copies of a book is produced from price and quantity.

4.2 DD S02 L02 - Entities, Instances, Attributes, Identifiers

An entity is a type of object stored in the database, for example `tds_book`. An instance is one concrete row, for example the book `Orbit of Tomorrow`. An attribute is a property such as `title`, `isbn`, or `price`. An identifier is a key that uniquely identifies an instance, for example `book_id` in `tds_book`.

4.3 DD S03 L01 - Relationships, Cardinality, Optionality

Relationship	Description
<code>tds_person - tds_customer</code>	One person may be one customer, and each customer must belong to exactly one person. The customer side is mandatory, while the person side is optional because not every person is a customer.
<code>tds_person - tds_employee</code>	One person may be one employee, and each employee must belong to exactly one person. The employee side is mandatory, while the person side is optional because not every person is an employee.
<code>tds_employee - tds_employee_salary_history</code>	One employee may have many salary history rows, and each salary history row must belong to exactly one employee. The history row cannot exist without an employee.
<code>tds_publisher - tds_book</code>	One publisher may publish many books, and each book must have exactly one publisher. The book membership in publisher is mandatory.
<code>tds_category - tds_book</code>	One category may contain many books, and each book must belong to exactly one category. The book membership in category is mandatory.
<code>tds_category - tds_category</code>	One category may have many child categories, and one child category may have one parent category. The parent category is optional because the root category has no parent.
<code>tds_book - tds_book_author</code>	One book must have at least one author in the business model, and one row in <code>tds_book_author</code> must reference exactly one book. The relationship row cannot exist without a book.
<code>tds_author - tds_book_author</code>	One author may write many books, and one row in <code>tds_book_author</code> must reference exactly one author. The relationship row cannot exist without an author.
<code>tds_customer - tds_customer_order</code>	One customer may place many orders, and each order must belong to exactly one customer. The order membership in customer is mandatory.
<code>tds_employee - tds_customer_order</code>	One employee may process many orders, and one order may be processed by one employee. The employee is optional because a new order may not yet be assigned.
<code>tds_customer_order - tds_order_item</code>	One order may contain many order items, and each order item must belong to exactly one order. The order item cannot exist without its parent order.
<code>tds_book - tds_order_item</code>	One book may appear in many order items, and each order item must reference exactly one book. The order item membership in book is mandatory.

tds_customer_order - tds_payment	One order may have one or more payments, and each payment must belong to exactly one order. The payment cannot exist without an order.
---	--

4.4 DD S03 L02 - ER Diagram

The ER diagram is created in Oracle SQL Developer Data Modeler. The image placeholder is in the section *Model Images*.

4.5 DD S30 L04 - Relationship Matrix Diagram

	Person	Customer	Employee	Book	Author	Category	Order	Payment
Person		subtype	subtype					
Customer	subtype						1:N	
Employee	subtype						1:N optional	
Book					M:N	N:1	M:N via item	
Author				M:N				
Category				1:N		recursive		
Order		N:1	optional N:1	M:N via item				1:N
Payment							N:1	

4.6 DD S04 L01 - Supertypes and Subtypes

tds_person is the supertype. **tds_customer** and **tds_employee** are subtypes. The discriminator is **person_type**, where C means customer and E means employee.

4.7 DD S04 L02 - Business Rules

Every customer and employee must be a person. A person email must be unique and must have a valid format. Every book must have one publisher and one category. An order must belong to one customer. An order item must belong to one order and one book. Payment method fields are mutually exclusive according to the payment method.

4.8 DD S05 L01 - Transferable and Non-transferable Relationship

Transferable relationship: an order may be assigned to another employee, so **tds_customer_order.employee_id** is transferable. Non-transferable relationship: an order item belongs to its original order, and its key contains **order_id**, so it should not be moved to another order.

4.9 DD S05 L03 - M:N Relationships

M:N without information: **tds_book_author** connects books and authors and stores only keys. M:N with information: **tds_order_item** connects orders and books and stores **quantity**, **unit_price**, and **discount_pct**.

4.10 DD S06 L01 - Identifying 1:N Relationship

tds_customer_order to **tds_order_item** is identifying. The child primary key is (**order_id**, **line_no**), and **order_id** is also a foreign key to **tds_customer_order**.

4.11 DD S06 L02-04 - Normal Forms

First normal form: all attributes are atomic. Second normal form: non-key attributes in composite-key tables depend on the full key. Third normal form: secondary attributes do not depend on other secondary attributes.

4.12 DD S07 L01 - ARC

The ARC-like design is in **tds_payment**. Exactly one method-specific code must be filled according to **payment_method**: card, bank transfer, or gift card.

4.13 DD S07 L02 - Hierarchical and Recursive Relations

The recursive relation is `tds_category.parent_category_id`, which references `tds_category.category_id`. It creates a tree of book categories.

4.14 DD S07 L03 - Historical Data

Historical salary data are stored in `tds_employee_salary_history`. Each row has `valid_from`, `valid_to`, `old_salary`, `new_salary`, and `changed_at`.

4.15 DD S09 L01 - Storing Changes in Time

Changes over time are represented by multiple salary history rows for the same employee. The current salary row has `valid_to` IS NULL.

4.16 DD S09 L02 - Journaling

The salary history table is a journal table. It stores previous and new salary values, validity dates, and the reason for change.

4.17 DD S10 L01 - Readability Conventions

The schema uses the prefix `tds_`, singular entity names, named constraints, readable key names, and consistent foreign key names.

4.18 DD S10 L02 - Generic Modeling

The project mostly uses traditional modeling because books, orders, and payments have clear attributes. A limited generic approach is used in `tds_payment`, where `payment_method` decides which method-specific code is valid.

4.19 DD S11 L01 - Integrity Constraints

Entity integrity is implemented by primary keys. Relationship integrity is implemented by foreign keys. Attribute integrity is implemented by NOT NULL, UNIQUE, and CHECK constraints. User-defined integrity is implemented by triggers that enforce correct person subtype rows.

4.20 DD S11 L02-04 - Relational Schema Generation

The conceptual model is transformed into relational tables. M:N relationships become intersection tables, subtypes become separate subtype tables with shared primary keys, and recursive relationships become self-referencing foreign keys.

5 SQL Requirements

Each point below has a direct Oracle SQL example. The same examples are also in `tds_bookstore_demo_queries_each_questi` which should be run after the create and insert scripts.

5.1 DD S15 L01 - string concatenation with ||, CONCAT(), DISTINCT

```
SELECT first_name || ' ' || last_name AS full_name FROM tds_person;
SELECT CONCAT(CONCAT(first_name, ' '), last_name) AS full_name FROM tds_person;
SELECT DISTINCT order_status FROM tds_customer_order;
```

5.2 DD S16 L02 - WHERE, LOWER, UPPER, INITCAP

```
SELECT book_id, title, price FROM tds_book WHERE price > 300;
SELECT LOWER(email) AS lower_email, UPPER(last_name) AS upper_last_name, INITCAP(first_name) AS initcap_first_name FROM tds_person;
```

5.3 DD S16 L03 - BETWEEN, LIKE, IN, IS NULL, IS NOT NULL

```
SELECT title, price FROM tds_book WHERE price BETWEEN 300 AND 500;
SELECT title FROM tds_book WHERE title LIKE '%Data%';
SELECT order_id, order_status FROM tds_customer_order WHERE order_status IN ('NEW','PAID','SHIPPED');
SELECT order_id FROM tds_customer_order WHERE delivery_note IS NULL;
SELECT order_id FROM tds_customer_order WHERE delivery_note IS NOT NULL;
```

5.4 DD S17 L01 - AND, OR, NOT, parentheses

```
SELECT title, price, page_count FROM tds_book
WHERE (price > 300 AND page_count > 200) OR NOT category_id = 5;
```

5.5 DD S17 L02 - ORDER BY ASC/DESC and multiple columns

```
SELECT title, price, page_count FROM tds_book ORDER BY price DESC;
SELECT title, price, page_count FROM tds_book ORDER BY price DESC, title ASC;
```

5.6 DD S17 L03 - single-row and multi-row functions

```
SELECT title, ROUND(price * 1.21, 2) AS price_with_vat FROM tds_book;
SELECT MIN(price), MAX(price), AVG(price), SUM(price), COUNT(*) FROM tds_book;
```

5.7 SQL S01 L01 - character functions and DUAL

```
SELECT LOWER('BOOKSTORE') AS lower_text,
UPPER('bookstore') AS upper_text,
INITCAP('online bookstore') AS initcap_text,
CONCAT('Book', 'Store') AS concat_text,
SUBSTR('BookStore', 1, 4) AS substr_text,
LENGTH('BookStore') AS text_length,
INSTR('BookStore', 'Store') AS instr_pos,
LPAD('42', 5, '0') AS lpad_text,
RPAD('Book', 8, '.') AS rpad_text,
TRIM(' Book ') AS trimmed_text,
REPLACE('Old Book', 'Old', 'New') AS replaced_text
FROM dual;
```

5.8 SQL S01 L02 - ROUND, TRUNC, MOD

```
SELECT title, price,
ROUND(price, 2) AS rounded_2,
TRUNC(price, 0) AS truncated_integer,
ROUND(price, -3) AS rounded_thousands,
MOD(page_count, 2) AS page_count_mod_2
FROM tds_book;
```

5.9 SQL S01 L03 - date functions and SYSDATE

```
SELECT MONTHS_BETWEEN(SYSDATE, registered_at) AS months_registered,
ADD_MONTHS(registered_at, 6) AS six_months_later,
NEXT_DAY(registered_at, 'MONDAY') AS next_monday,
LAST_DAY(registered_at) AS month_end,
ROUND(registered_at, 'MONTH') AS rounded_month,
TRUNC(registered_at, 'MONTH') AS first_day_of_month
FROM tds_customer;
```

5.10 SQL S02 L01 - TO_CHAR, TO_NUMBER, TO_DATE

```
SELECT TO_CHAR(order_date, 'YYYY-MM-DD') AS order_date_text FROM tds_customer_order;
SELECT TO_NUMBER('1234.50') + 10 AS number_result FROM dual;
SELECT TO_DATE('2026-05-14', 'YYYY-MM-DD') AS converted_date FROM dual;
```

5.11 SQL S02 L02 - NVL, NVL2, NULLIF, COALESCE

```
SELECT order_id,
       NVL(DBMS_LOB.SUBSTR(delivery_note, 4000, 1), 'No note') AS note_text,
       NVL2(employee_id, 'Assigned', 'Unassigned') AS employee_assignment,
       NULLIF(order_status, 'NEW') AS status_unless_new,
       COALESCE(DBMS_LOB.SUBSTR(delivery_note, 4000, 1), shipping_address, 'No text') AS first_available_text
FROM tds_customer_order;
```

5.12 SQL S02 L03 - DECODE, CASE, IF-THEN-ELSE

```
SELECT order_id,
       DECODE(order_status, 'NEW', 'Open', 'PAID', 'Paid', 'Other') AS decoded_status,
       CASE WHEN total_amount >= 800 THEN 'High value' ELSE 'Normal value' END AS value_group
FROM tds_customer_order;
DECLARE
  v_count NUMBER;
BEGIN
  SELECT COUNT(*) INTO v_count FROM tds_customer_order;
  IF v_count > 0 THEN
    DBMS_OUTPUT.PUT_LINE('There are orders in the system.');
```

5.13 SQL S03 L01 - NATURAL JOIN, CROSS JOIN

```
SELECT order_id, line_no, quantity FROM tds_customer_order NATURAL JOIN tds_order_item;
SELECT p.publisher_name, c.category_name FROM tds_publisher p CROSS JOIN tds_category c WHERE ROWNUM <= 10;
```

5.14 SQL S03 L02 - JOIN USING and JOIN ON

```
SELECT order_id, line_no, quantity FROM tds_customer_order JOIN tds_order_item USING (order_id);
SELECT b.title, p.publisher_name FROM tds_book b JOIN tds_publisher p ON b.publisher_id = p.publisher_id;
```

5.15 SQL S03 L03 - OUTER JOIN examples

```
SELECT b.title, oi.order_id FROM tds_book b LEFT OUTER JOIN tds_order_item oi ON b.book_id = oi.book_id;
SELECT b.title, oi.order_id FROM tds_order_item oi RIGHT OUTER JOIN tds_book b ON oi.book_id = b.book_id;
SELECT b.title, oi.order_id FROM tds_book b FULL OUTER JOIN tds_order_item oi ON b.book_id = oi.book_id;
```

5.16 SQL S03 L04 - self join and hierarchical query on recursive category table

```
SELECT child.category_name AS child_category, parent.category_name AS parent_category
FROM tds_category child LEFT JOIN tds_category parent ON child.parent_category_id = parent.category_id;
SELECT LEVEL AS category_level, LPAD(' ', 2 * (LEVEL - 1)) || category_name AS category_tree
FROM tds_category
START WITH parent_category_id IS NULL
CONNECT BY PRIOR category_id = parent_category_id
ORDER SIBLINGS BY category_name;
```

5.17 SQL S04 L02 - aggregate functions

```
SELECT AVG(price), COUNT(*), MIN(price), MAX(price), SUM(price), VARIANCE(price), STDDEV(price) FROM tds_book;
```

5.18 SQL S04 L03 - COUNT variants and NVL

```
SELECT COUNT(*) AS all_orders, COUNT(employee_id) AS assigned_orders, COUNT(DISTINCT customer_id) AS distinct_customers FROM  
    tds_customer_order;  
SELECT AVG(NVL(discount_pct, 0)) AS avg_discount_with_null_as_zero FROM tds_order_item;
```

5.19 SQL S05 L01 - GROUP BY and HAVING

```
SELECT customer_id, COUNT(*) AS order_count, SUM(total_amount) AS total_spent  
FROM tds_customer_order  
GROUP BY customer_id  
HAVING SUM(total_amount) > 400;
```

5.20 SQL S05 L02 - ROLLUP, CUBE, GROUPING SETS

```
SELECT order_status, employee_id, SUM(total_amount) AS total_amount  
FROM tds_customer_order  
GROUP BY ROLLUP(order_status, employee_id);  
SELECT order_status, employee_id, SUM(total_amount) AS total_amount  
FROM tds_customer_order  
GROUP BY CUBE(order_status, employee_id);  
SELECT order_status, employee_id, SUM(total_amount) AS total_amount  
FROM tds_customer_order  
GROUP BY GROUPING SETS ((order_status), (employee_id), ());
```

5.21 SQL S05 L03 - UNION, UNION ALL, INTERSECT, MINUS

```
SELECT email FROM tds_person WHERE person_type = 'C'  
UNION  
SELECT email FROM tds_person WHERE person_type = 'E'  
ORDER BY email;  
SELECT email FROM tds_person WHERE person_type = 'C'  
UNION ALL  
SELECT email FROM tds_person WHERE person_type = 'E';  
SELECT person_id FROM tds_person  
INTERSECT  
SELECT customer_id FROM tds_customer;  
SELECT person_id FROM tds_person  
MINUS  
SELECT customer_id FROM tds_customer;
```

5.22 SQL S06 L01 - subqueries: scalar, multi-column, EXISTS, NOT EXISTS

```
SELECT title, price FROM tds_book WHERE price > (SELECT AVG(price) FROM tds_book);  
SELECT book_id, title, price FROM tds_book  
WHERE (category_id, publisher_id) IN (SELECT category_id, publisher_id FROM tds_book WHERE price > 400);  
SELECT c.customer_id FROM tds_customer c WHERE EXISTS (SELECT 1 FROM tds_customer_order o WHERE o.customer_id = c.customer_id);  
SELECT c.customer_id FROM tds_customer c WHERE NOT EXISTS (SELECT 1 FROM tds_customer_order o WHERE o.customer_id = c.customer_id);
```

5.23 SQL S06 L02 - single-row subquery

```
SELECT title, price FROM tds_book WHERE price = (SELECT MAX(price) FROM tds_book);
```

5.24 SQL S06 L03 - multi-row subqueries IN, ANY, ALL and NULL comment

```
SELECT title FROM tds_book WHERE category_id IN (SELECT category_id FROM tds_category WHERE parent_category_id IS NOT NULL);
SELECT title, price FROM tds_book WHERE price > ANY (SELECT unit_price FROM tds_order_item);
SELECT title, price FROM tds_book WHERE price >= ALL (SELECT unit_price FROM tds_order_item WHERE unit_price IS NOT NULL);
```

5.25 SQL S06 L04 - WITH AS

```
WITH order_totals AS (
  SELECT customer_id, SUM(total_amount) AS total_spent
  FROM tds_customer_order
  GROUP BY customer_id
)
SELECT customer_id, total_spent FROM order_totals WHERE total_spent > 400;
```

5.26 SQL S07 L01 - INSERT forms

```
CREATE TABLE tds_demo_people (person_id NUMBER(10), full_name VARCHAR2(120));
INSERT INTO tds_demo_people VALUES (1, 'Manual Person');
INSERT INTO tds_demo_people (person_id, full_name) VALUES (2, 'Named Columns Person');
INSERT INTO tds_demo_people (person_id, full_name)
SELECT person_id, first_name || ' ' || last_name FROM tds_person WHERE person_id <= 3;
CREATE TABLE tds_demo_expensive_books AS SELECT book_id, title, price FROM tds_book WHERE price > 300;
```

5.27 SQL S07 L02 - UPDATE and DELETE

```
UPDATE tds_demo_people SET full_name = INITCAP(full_name) WHERE person_id = 1;
DELETE FROM tds_demo_people WHERE person_id = 2;
```

5.28 SQL S07 L03 - DEFAULT, MERGE, multi-table insert

```
CREATE TABLE tds_demo_stock (book_id NUMBER(10) PRIMARY KEY, stock_qty NUMBER(8) DEFAULT 0 NOT NULL);
INSERT INTO tds_demo_stock (book_id) VALUES (1);
MERGE INTO tds_demo_stock dst
USING (SELECT 1 AS book_id, 12 AS stock_qty FROM dual) src
ON (dst.book_id = src.book_id)
WHEN MATCHED THEN UPDATE SET dst.stock_qty = src.stock_qty
WHEN NOT MATCHED THEN INSERT (book_id, stock_qty) VALUES (src.book_id, src.stock_qty);
CREATE TABLE tds_demo_a (id NUMBER, label VARCHAR2(20));
CREATE TABLE tds_demo_b (id NUMBER, label VARCHAR2(20));
INSERT ALL
  INTO tds_demo_a (id, label) VALUES (book_id, 'A')
  INTO tds_demo_b (id, label) VALUES (book_id, 'B')
SELECT book_id FROM tds_book WHERE book_id <= 2;
```

5.29 SQL S08 L01 - objects: table, index, constraint, view, sequence, synonym; CREATE/ALTER/DROP/RENAME/TRUNCATE

```
CREATE TABLE tds_demo_alter (id NUMBER CONSTRAINT pk_tds_demo_alter PRIMARY KEY, name VARCHAR2(30) DEFAULT 'N/A' NOT NULL);
CREATE INDEX idx_tds_demo_alter_name ON tds_demo_alter(name);
CREATE OR REPLACE VIEW tds_v_order_totals AS SELECT order_status, SUM(total_amount) AS total_amount FROM tds_customer_order GROUP
  BY order_status;
CREATE SEQUENCE tds_demo_seq START WITH 100 INCREMENT BY 5 NOCACHE NOCYCLE;
CREATE SYNONYM tds_books_synonym FOR tds_book;
ALTER TABLE tds_demo_alter ADD created_at TIMESTAMP DEFAULT SYSTIMESTAMP NOT NULL;
CREATE TABLE tds_demo_rename_old (id NUMBER);
RENAME tds_demo_rename_old TO tds_demo_rename_new;
TRUNCATE TABLE tds_demo_rename_new;

-- SQL S08 L01 optional external table template - commented because DIRECTORY privilege may be unavailable
-- CREATE TABLE tds_demo_external_books (
--   title VARCHAR2(200), price NUMBER(10,2)
-- ) ORGANIZATION EXTERNAL (
--   TYPE ORACLE_LOADER
--   DEFAULT DIRECTORY DATA_PUMP_DIR
```

```
-- ACCESS PARAMETERS (
--   RECORDS DELIMITED BY NEWLINE
--   FIELDS TERMINATED BY ','
-- )
-- LOCATION ('books.csv')
-- );
```

5.30 SQL S08 L02 - datatype demo: timestamps, intervals, text, number, blob

```
CREATE TABLE tds_demo_types (
  id NUMBER,
  fixed_code CHAR(2),
  variable_text VARCHAR2(100),
  long_text CLOB,
  binary_data BLOB,
  created_at TIMESTAMP DEFAULT SYSTIMESTAMP,
  created_at_tz TIMESTAMP WITH TIME ZONE,
  created_at_local TIMESTAMP WITH LOCAL TIME ZONE,
  revision_period INTERVAL YEAR TO MONTH,
  reading_period INTERVAL DAY TO SECOND
);
INSERT INTO tds_demo_types VALUES (
  1,
  'CZ',
  'demo',
  'clob text',
  TO_BLOB(UTL_RAW.CAST_TO_RAW('blob text')),
  SYSTIMESTAMP,
  SYSTIMESTAMP,
  SYSTIMESTAMP,
  INTERVAL '1-2' YEAR TO MONTH,
  INTERVAL '0 02:30:00' DAY TO SECOND
);
SELECT id, fixed_code, variable_text, created_at, created_at_tz, created_at_local,
       revision_period, reading_period
FROM tds_demo_types;
```

5.31 SQL S08 L03 - ALTER TABLE, DROP, RENAME, FLASHBACK, DELETE, TRUNCATE, COMMENT, SET UNUSED

```
ALTER TABLE tds_demo_alter MODIFY name VARCHAR2(60);
ALTER TABLE tds_demo_alter ADD note VARCHAR2(100);
ALTER TABLE tds_demo_alter DROP COLUMN note;
COMMENT ON TABLE tds_demo_alter IS 'Demo table for ALTER and COMMENT commands.';
CREATE TABLE tds_demo_unused (id NUMBER, temporary_text VARCHAR2(50));
ALTER TABLE tds_demo_unused SET UNUSED (temporary_text);
DELETE FROM tds_demo_a WHERE id = 1;
TRUNCATE TABLE tds_demo_b;
CREATE TABLE tds_demo_flashback (id NUMBER);
DROP TABLE tds_demo_flashback;
FLASHBACK TABLE tds_demo_flashback TO BEFORE DROP;
DROP TABLE tds_demo_flashback PURGE;
```

5.32 SQL S10 L01 - named constraints and CTAS

```
CREATE TABLE tds_demo_constraints (
  id NUMBER CONSTRAINT pk_tds_demo_constraints PRIMARY KEY,
  code VARCHAR2(20) CONSTRAINT nn_tds_demo_constraints_code NOT NULL,
  email VARCHAR2(100),
  CONSTRAINT uq_tds_demo_constraints_email UNIQUE (email)
);
CREATE TABLE tds_demo_book_copy AS SELECT book_id, title, price FROM tds_book;
DROP TABLE tds_demo_book_copy PURGE;
DROP TABLE tds_demo_constraints PURGE;
```

5.33 SQL S10 L02 - constraints including FK and CHECK

```
CREATE TABLE tds_demo_parent (id NUMBER CONSTRAINT pk_tds_demo_parent PRIMARY KEY);
CREATE TABLE tds_demo_child (
  id NUMBER CONSTRAINT pk_tds_demo_child PRIMARY KEY,
  parent_id NUMBER CONSTRAINT nn_tds_demo_child_parent NOT NULL,
  amount NUMBER CONSTRAINT ck_tds_demo_child_amount CHECK (amount >= 0),
  CONSTRAINT fk_tds_demo_child_parent FOREIGN KEY (parent_id) REFERENCES tds_demo_parent(id) ON DELETE CASCADE
);
DROP TABLE tds_demo_child PURGE;
DROP TABLE tds_demo_parent PURGE;
```

5.34 SQL S10 L03 - USER_CONSTRAINTS

```
SELECT constraint_name, constraint_type, table_name FROM user_constraints WHERE table_name LIKE 'TDS_%' ORDER BY table_name,
constraint_name;
```

5.35 SQL S11 L01 - views: FORCE, NOFORCE, WITH CHECK OPTION, READ ONLY, simple and complex

```
CREATE OR REPLACE FORCE VIEW tds_v_force_demo AS SELECT book_id, title, price FROM tds_book;
CREATE OR REPLACE NOFORCE VIEW tds_v_noforce_demo AS SELECT person_id, email FROM tds_person;
CREATE OR REPLACE VIEW tds_v_book_prices AS SELECT book_id, title, price FROM tds_book WHERE price >= 0 WITH CHECK OPTION;
CREATE OR REPLACE VIEW tds_v_readonly_books AS SELECT book_id, title, price FROM tds_book WITH READ ONLY;
CREATE OR REPLACE VIEW tds_v_order_totals AS
SELECT o.order_id, p.first_name || ' ' || p.last_name AS customer_name, SUM(oi.quantity * oi.unit_price) AS calculated_total
FROM tds_customer_order o
JOIN tds_customer c ON o.customer_id = c.customer_id
JOIN tds_person p ON c.customer_id = p.person_id
JOIN tds_order_item oi ON o.order_id = oi.order_id
GROUP BY o.order_id, p.first_name, p.last_name;
```

5.36 SQL S11 L03 - inline view

```
SELECT title, price FROM (SELECT title, price FROM tds_book WHERE price > 300) expensive_books ORDER BY price DESC;
```

5.37 SQL S12 L01 - CREATE and ALTER SEQUENCE

```
CREATE SEQUENCE tds_demo_seq_s12
  START WITH 10
  INCREMENT BY 2
  NOMAXVALUE
  NOMINVALUE
  NOCYCLE
  NOCACHE;
SELECT tds_demo_seq_s12.NEXTVAL AS demo_sequence_value FROM dual;
ALTER SEQUENCE tds_demo_seq_s12 INCREMENT BY 1 NOCACHE NOCYCLE;
SELECT tds_demo_seq_s12.NEXTVAL AS demo_sequence_value_after_alter FROM dual;
```

5.38 SQL S12 L02 - index and key objects

```
SELECT index_name, table_name, uniqueness FROM user_indexes WHERE table_name LIKE 'TDS_%' ORDER BY table_name, index_name;
```

5.39 SQL S13 L01 - GRANT and REVOKE on a view to PUBLIC

```
CREATE OR REPLACE VIEW tds_v_grant_demo AS SELECT book_id, title FROM tds_book;
GRANT SELECT ON tds_v_grant_demo TO PUBLIC;
REVOKE SELECT ON tds_v_grant_demo FROM PUBLIC;
```

5.40 SQL S13 L03 - regular expressions

```
SELECT email FROM tds_person WHERE REGEXP_LIKE(email, '^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$');
SELECT REGEXP_REPLACE(phone, '[^0-9]', '') AS digits_only FROM tds_person WHERE phone IS NOT NULL;
SELECT REGEXP_INSTR(email, '@') AS at_position FROM tds_person;
SELECT REGEXP_SUBSTR(email, '[^@]+', 1, 1) AS email_user_name FROM tds_person;
SELECT REGEXP_COUNT(title, '[aeiouAEIOU]') AS vowel_count, title FROM tds_book;
```

5.41 SQL S14 L01 - transaction commands

```
SAVEPOINT before_demo_update;
UPDATE tds_demo_stock SET stock_qty = stock_qty + 1 WHERE book_id = 1;
ROLLBACK TO before_demo_update;
COMMIT;
```

5.42 SQL S15 L01 - old join syntax and old outer join syntax

```
SELECT b.title, p.publisher_name FROM tds_book b, tds_publisher p WHERE b.publisher_id = p.publisher_id;
SELECT b.title, oi.order_id FROM tds_book b, tds_order_item oi WHERE b.book_id = oi.book_id(+);
```

5.43 SQL S16 L03 - recap query combining several concepts

```
WITH customer_spending AS (
  SELECT o.customer_id, SUM(o.total_amount) AS total_spent, COUNT(*) AS order_count
  FROM tds_customer_order o
  GROUP BY o.customer_id
)
SELECT p.first_name || ' ' || p.last_name AS customer_name,
       NVL(cs.total_spent, 0) AS total_spent,
       CASE WHEN NVL(cs.total_spent, 0) > 800 THEN 'VIP' ELSE 'STANDARD' END AS customer_group
FROM tds_customer c
JOIN tds_person p ON c.customer_id = p.person_id
LEFT JOIN customer_spending cs ON c.customer_id = cs.customer_id
ORDER BY total_spent DESC;
```